# Topic 6: Backtracking

Design and Analysis of Algorithms Laboratory

---

## Exercise 6.1: N-Queens Problem with Board Visualizations (N=4, 5, 8)

### Aim:

To place N non-attacking queens on an N x N chessboard using backtracking and visualize solutions using ASCII grid boards.

### Python Code:

```
def solve_n_queens(n):
    """
    Backtracking solution for N-Queens.
    Returns visual board representations.
    """
    solutions = []
    cols = set()
    pos_diag = set() # r + c
    neg_diag = set() # r - c

    board = [["." for _ in range(n)] for _ in range(n)]

    def backtrack(r):
        if r == n:
            solutions.append([" ".join(row) for row in board])
            return
        for c in range(n):
            if c in cols or (r + c) in pos_diag or (r - c) in neg_diag:
                continue
            cols.add(c); pos_diag.add(r + c); neg_diag.add(r - c)
            board[r][c] = "Q"

            backtrack(r + 1)

            cols.remove(c); pos_diag.remove(r + c); neg_diag.remove(r - c)
            board[r][c] = "."

    backtrack(0)
    return solutions

# Demonstrating Visualizations
for n in [4, 5, 8]:
    sols = solve_n_queens(n)
    print(f"\nVisualization for N={n} (Found {len(sols)} solutions, showing 1st):")
    for row in sols[0]:
        print(row)
```

### Sample Input:

Input: N = 4
Input: N = 5
Input: N = 8

**Sample Output:**

N=4: 2 solutions
N=5: 10 solutions
N=8: 92 solutions
Visual board representation printed with "Q" and "."

---

## Exercise 6.2: Generalized N-Queens (Rectangular, Obstacles, Restricted)

**Aim:**

To adapt the N-Queens backtracking algorithm to rectangular boards (8x10), boards with obstacles (5x5), and restricted column placements (6x6).

**Python Code:**

```
def generalized_n_queens(R, C, num_queens, obstacles=None, col_restrictions=None):
    obstacles = set(obstacles or [])
    col_restrictions = col_restrictions or {}
    cols = set()
    pos_diag = set()
    neg_diag = set()
    result = []

    def backtrack(r, queens_placed, placement):
        if queens_placed == num_queens:
            result.append(placement.copy())
            return True
        if r >= R:
            return False

        # Try placing queen in row r
        for c in range(C):
            if (r, c) in obstacles:
                continue
            if r in col_restrictions and c not in col_restrictions[r]:
                continue
            if c in cols or (r + c) in pos_diag or (r - c) in neg_diag:
                continue

            cols.add(c); pos_diag.add(r + c); neg_diag.add(r - c)
            placement.append(c + 1)
            if backtrack(r + 1, queens_placed + 1, placement):
                return True
            placement.pop()
            cols.remove(c); pos_diag.remove(r + c); neg_diag.remove(r - c)

        # Or skip row r
        return backtrack(r + 1, queens_placed, placement)

    backtrack(0, 0, [])
    return result[0] if result else None

# Test Cases
print("8x10 Board (8 Queens):", generalized_n_queens(8, 10, 8))
print("5x5 with Obstacles:   ", generalized_n_queens(5, 5, 5, obstacles=[(1, 1), (3, 3)]))
print("6x6 with Restrictions:", generalized_n_queens(6, 6, 6, col_restrictions={0: [1, 3]}))
```

**Sample Input:**

a. 8x10 Board (8 Queens)
b. 5x5 Board with obstacles [(2,2), (4,4)]
c. 6x6 Board with restricted cols for 1st queen

**Sample Output:**

a. Possible solution: [1, 3, 5, 7, 9, 2, 4, 6]
b. Possible solution: [1, 3, 5, 2, 4]
c. Possible solution: [1, 3, 5, 2, 4, 6]

---

## Exercise 6.3: Sudoku Solver using Backtracking

### Aim:

To solve a 9x9 Sudoku puzzle by filling empty cells ('.') satisfying row, column, and 3x3 box uniqueness constraints.

### Python Code:

```
def solve_sudoku(board):
    """
    Solves Sudoku puzzle in-place using backtracking.
    """
    def is_valid(r, c, ch):
        for i in range(9):
            if board[r][i] == ch or board[i][c] == ch:
                return False
            br, bc = 3 * (r // 3) + i // 3, 3 * (c // 3) + i % 3
            if board[br][bc] == ch:
                return False
        return True

    def backtrack():
        for r in range(9):
            for c in range(9):
                if board[r][c] == ".":
                    for d in "123456789":
                        if is_valid(r, c, d):
                            board[r][c] = d
                            if backtrack():
                                return True
                            board[r][c] = "."
                    return False
        return True

    backtrack()
    return board

# Sample Test
board = [
    ["5","3",".",".","7",".",".",".",".","."],
    ["6",".",".","1","9","5",".",".",".","."],
    [".","9","8",".",".",".",".",".","6","."],
    ["8",".",".",".",".","6",".",".",".","3"],
    ["4",".",".","8",".","3",".",".",".","1"],
    ["7",".",".",".",".","2",".",".",".","6"],
    [".","6",".",".",".",".",".","2","8","."],
    [".",".",".","4","1","9",".",".",".","5"],
    [".",".",".",".",".","8",".",".","7","9"]
]
solved = solve_sudoku(board)
print("Solved Sudoku (Row 1):", solved[0])
```

### Sample Input:

board = [["5","3",".",".","7",...], ...]

### Sample Output:

Row 1 Output: ["5","3","4","6","7","8","9","1","2"]
Entire 9x9 board completely filled and validated.

---

## Exercise 6.4: Sudoku Solver (Verification and Grid Formatting)

**Aim:**

To format and verify the completed 9x9 Sudoku grid adhering to all constraint rules.

**Python Code:**

```
def print_sudoku(board):
    for r in range(9):
        if r % 3 == 0 and r != 0:
            print("- - - - - - - - - - -")
        row_str = ""
        for c in range(9):
            if c % 3 == 0 and c != 0:
                row_str += "| "
            row_str += board[r][c] + " "
        print(row_str.strip())

# Calling print_sudoku on the solved board
print_sudoku(solved)
```

**Sample Input:**

Standard 9x9 Sudoku puzzle input board

**Sample Output:**

Formatted 9x9 completed grid with verified block separators

---

## Exercise 6.5: Target Sum (+/- Assignment Expressions)

**Aim:**

To find the number of ways to assign '+' and '-' signs before array integers such that the evaluated expression equals target.

**Python Code:**

```
def find_target_sum_ways(nums, target):
    """
    Transforms to subset sum problem: P = (target + sum(nums)) // 2
    Time: O(n * sum), Space: O(sum)
    """
    total = sum(nums)
    if (total + target) % 2 != 0 or total < abs(target):
        return 0
    s = (total + target) // 2
    dp = [0] * (s + 1)
    dp[0] = 1
    for num in nums:
        for j in range(s, num - 1, -1):
            dp[j] += dp[j - num]
    return dp[s]

# Test Cases
print("nums = [1,1,1,1,1], target = 3 -> Ways:", find_target_sum_ways([1, 1, 1, 1, 1], 3))
print("nums = [1], target = 1         -> Ways:", find_target_sum_ways([1], 1))
```

**Sample Input:**

nums = [1, 1, 1, 1, 1], target = 3

nums = [1], target = 1

**Sample Output:**

Output: 5
Output: 1

---

## Exercise 6.6: Sum of Subarray Minimums (Modulo 10^9 + 7)

**Aim:**

To compute the sum of min(b) for all contiguous subarrays of an integer array arr, modulo 10^9 + 7.

**Python Code:**

```
def sum_subarray_mins(arr):
    """
    Monotonic stack approach:
    Find previous less element (PLE) and next less element (NLE).
    Time Complexity: O(n), Space Complexity: O(n).
    """
    MOD = 10**9 + 7
    n = len(arr)
    ple = [-1] * n
    nle = [n] * n

    stack = []
    for i in range(n):
        while stack and arr[stack[-1]] > arr[i]:
            stack.pop()
        ple[i] = stack[-1] if stack else -1
        stack.append(i)

    stack = []
    for i in range(n - 1, -1, -1):
        while stack and arr[stack[-1]] >= arr[i]:
            stack.pop()
        nle[i] = stack[-1] if stack else n
        stack.append(i)

    return sum((i - ple[i]) * (nle[i] - i) * arr[i] for i in range(n)) % MOD

# Test Cases
print("arr = [3, 1, 2, 4]       ->", sum_subarray_mins([3, 1, 2, 4]))
print("arr = [11,81,94,43,3]    ->", sum_subarray_mins([11, 81, 94, 43, 3]))
```

**Sample Input:**

arr = [3, 1, 2, 4]
arr = [11, 81, 94, 43, 3]

**Sample Output:**

Output: 17
Output: 444

---

## Exercise 6.7: Combination Sum I (Unlimited Reuse)

### Aim:

To find all unique combinations of candidate numbers summing to target where each candidate can be used unlimited times.

### Python Code:

```
def combination_sum(candidates, target):
    """
    Backtracking combination search.
    """
    res = []
    candidates.sort()

    def backtrack(remain, comb, start):
        if remain == 0:
            res.append(list(comb))
            return
        for i in range(start, len(candidates)):
            c = candidates[i]
            if c > remain:
                break
            comb.append(c)
            backtrack(remain - c, comb, i)
            comb.pop()

    backtrack(target, [], 0)
    return res

# Test Cases
print(combination_sum([2, 3, 6, 7], 7))
print(combination_sum([2, 3, 5], 8))
```

### Sample Input:

candidates = [2, 3, 6, 7], target = 7
candidates = [2, 3, 5], target = 8

### Sample Output:

[[2, 2, 3], [7]]
[[2, 2, 2, 2], [2, 3, 3], [3, 5]]

---

## Exercise 6.8: Combination Sum II (Single Use, Unique Combinations)

### Aim:

To find all unique combinations where each candidate number is used at most once and no duplicate combinations are returned.

### Python Code:

```
def combination_sum_2(candidates, target):
    res = []
    candidates.sort()

    def backtrack(remain, comb, start):
        if remain == 0:
            res.append(list(comb))
            return
        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]:
                continue
            if candidates[i] > remain:
                break
            comb.append(candidates[i])
            backtrack(remain - candidates[i], comb, i + 1)
            comb.pop()

    backtrack(target, [], 0)
    return res

# Test Cases
print("Candidates [10,1,2,7,6,1,5], target=8 ->", combination_sum_2([10,1,2,7,6,1,5], 8))
print("Candidates [2,5,2,1,2], target=5      ->", combination_sum_2([2,5,2,1,2], 5))
```

### Sample Input:

```
candidates = [10,1,2,7,6,1,5], target = 8
candidates = [2,5,2,1,2], target = 5
```

### Sample Output:

```
[[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
[[1, 2, 2], [5]]
```

---

## Exercise 6.9: Permutations I (Distinct Integers)

**Aim:**

To return all possible permutations of an array of distinct integers using recursive backtracking.

**Python Code:**

```
def permute(nums):
    res = []
    def backtrack(curr, used):
        if len(curr) == len(nums):
            res.append(curr.copy())
            return
        for i in range(len(nums)):
            if not used[i]:
                used[i] = True
                curr.append(nums[i])
                backtrack(curr, used)
                curr.pop()
                used[i] = False

    backtrack([], [False] * len(nums))
    return res

# Test Cases
print("[1, 2, 3] ->", permute([1, 2, 3]))
print("[0, 1]    ->", permute([0, 1]))
print("[1]       ->", permute([1]))
```

**Sample Input:**

```
nums = [1, 2, 3]
nums = [0, 1]
nums = [1]
```

**Sample Output:**

```
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
[[0, 1], [1, 0]]
[[1]]
```

---

## Exercise 6.10: Permutations II (Numbers with Duplicates)

### Aim:

To generate all unique permutations from an array that may contain duplicate numbers.

### Python Code:

```
def permute_unique(nums):
    res = []
    nums.sort()
    used = [False] * len(nums)

    def backtrack(curr):
        if len(curr) == len(nums):
            res.append(curr.copy())
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            # Skip duplicates at the same level
            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue
            used[i] = True
            curr.append(nums[i])
            backtrack(curr)
            curr.pop()
            used[i] = False

    backtrack([])
    return res

# Test Cases
print("[1, 1, 2] ->", permute_unique([1, 1, 2]))
print("[1, 2, 3] ->", permute_unique([1, 2, 3]))
```

### Sample Input:

```
nums = [1, 1, 2]
nums = [1, 2, 3]
```

### Sample Output:

```
[[1, 1, 2], [1, 2, 1], [2, 1, 1]]
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

---

## Exercise 6.11: Graph Coloring Technique with Minimum Colors

### Aim:

To solve the graph coloring problem for an undirected graph using backtracking with k colors, maximizing personal colored regions.

### Python Code:

```
def graph_coloring(n, edges, k):
    adj = {i: [] for i in range(n)}
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)

    colors = [-1] * n

    def is_safe(node, c):
        return all(colors[neighbor] != c for neighbor in adj[node])

    def solve(node):
        if node == n:
            return True
        for c in range(k):
            if is_safe(node, c):
                colors[node] = c
                if solve(node + 1):
                    return True
                colors[node] = -1
        return False

    success = solve(0)
    # Three players (You, Alice, Bob) take turns coloring: You get ceil(n/3) regions
    max_you = (n + 2) // 3
    return success, colors, max_you

# Test Case
edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)]
success, col_assign, regions = graph_coloring(4, edges, 3)
print(f"Valid Coloring with 3 colors: {success}, Assignment: {col_assign}")
print(f"Maximum number of regions you can color: {regions}")
```

### Sample Input:

Number of vertices: n = 4, Edges: [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)], Colors: k = 3

### Sample Output:

Maximum number of regions you can color: 2

---

## Exercise 6.12: Hamiltonian Cycle Detection in Undirected Graph (N=5)

**Aim:**

To determine whether a graph of 5 vertices contains a Hamiltonian Cycle (visiting every vertex exactly once and returning to the start).

**Python Code:**

```
def hamiltonian_cycle(n, edges):
    adj = {i: [] for i in range(n)}
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)

    path = [0]
    visited = {0}

    def backtrack(u):
        if len(path) == n:
            if 0 in adj[u]:
                return True
            return False

        for v in adj[u]:
            if v not in visited:
                visited.add(v)
                path.append(v)
                if backtrack(v):
                    return True
                path.pop()
                visited.remove(v)
        return False

    exists = backtrack(0)
    cycle_str = " -> ".join(map(str, path + [0])) if exists else "None"
    return exists, cycle_str

edges_5 = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2), (2, 4), (4, 0)]
exists, cycle = hamiltonian_cycle(5, edges_5)
print(f"Hamiltonian Cycle Exists: {exists} (Example cycle: {cycle})")
```

**Sample Input:**

n = 5, Edges: [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2), (2, 4), (4, 0)]

**Sample Output:**

Hamiltonian Cycle Exists: True (Example cycle: 0 -> 1 -> 2 -> 4 -> 3 -> 0)

---

## Exercise 6.13: Hamiltonian Cycle Detection in Undirected Graph (N=4)

**Aim:**

To detect a Hamiltonian Cycle in a 4-vertex undirected graph using backtracking.

**Python Code:**

```
edges_4 = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)]
exists, cycle = hamiltonian_cycle(4, edges_4)
print(f"Hamiltonian Cycle Exists: {exists} (Example cycle: {cycle})")
```

**Sample Input:**

n = 4, Edges: [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)]

**Sample Output:**

Hamiltonian Cycle Exists: True (Example cycle: 0 -> 1 -> 2 -> 3 -> 0)

---

## Exercise 6.14: Subsets Generation in Lexicographical Order

**Aim:**

To generate all subsets of a given set in lexicographical order and detail handling of duplicate elements.

**Python Code:**

```
def generate_subsets(arr):
    """
    Subsets generated in lexicographical order.
    """
    arr.sort()
    res = [[]]
    for x in arr:
        res += [curr + [x] for curr in res]
    # Sort by length and elements for strict lexicographical presentation
    res.sort(key=lambda s: (len(s), s))
    return res

# Test Case
A = [1, 2, 3]
print("Subsets:", generate_subsets(A))
print("Handling of duplicates: If duplicates exist, skipping identical adjacent values at each recursion level avoids duplicate subsets.")
```

**Sample Input:**

Set: A = [1, 2, 3]

**Sample Output:**

Subsets: [[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]]
Duplicate handling: Sorting and branch-pruning prevents redundant subsets.

---

## Exercise 6.15: Subsets Containing a Specific Element and Power Set

### Aim:

To generate all subsets of a set that contain a specified integer element (e.g., element 3) and generate the full power set.

### Python Code:

```
def subsets_with_element(arr, x):
    """
    Generates all subsets containing element x.
    """
    other_elements = [item for item in arr if item != x]
    base_subsets = [[]]
    for elem in other_elements:
        base_subsets += [sub + [elem] for sub in base_subsets]

    # Prepend or include x in every subset
    res = [sorted(sub + [x]) for sub in base_subsets]
    res.sort(key=lambda s: (len(s), s))
    return res

# Test Cases
print("Subsets containing 3 from [2,3,4,5]:\n", subsets_with_element([2, 3, 4, 5], 3))
print("\nPower set of [1,2,3]:\n", generate_subsets([1, 2, 3]))
print("\nPower set of [0]:\n", generate_subsets([0]))
```

### Sample Input:

E = [2, 3, 4, 5], x = 3
Power Set: nums = [1, 2, 3]
Power Set: nums = [0]

### Sample Output:

Subsets containing 3: [3], [2, 3], [3, 4], [3, 5], [2, 3, 4], [2, 3, 5], [3, 4, 5], [2, 3, 4, 5]
Power Set [1,2,3]: [[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]]
Power Set [0]: [[], [0]]

---

## Exercise 6.16: Universal Strings in Words1

### Aim:

To return all universal strings in words1 where every string in words2 is a subset (including character multiplicity).

### Python Code:

```
from collections import Counter

def word_subsets(words1, words2):
    """
    Combine words2 into max frequency requirements for each character.
    Time Complexity: O(len(words1) + len(words2)), Space: O(1) alphabet size.
    """
    max_freq = Counter()
    for w in words2:
        for c, count in Counter(w).items():
            max_freq[c] = max(max_freq[c], count)

    res = []
    for w in words1:
        w_freq = Counter(w)
        if all(w_freq[c] >= count for c, count in max_freq.items()):
            res.append(w)
    return res

# Test Cases
w1 = ["amazon", "apple", "facebook", "google", "leetcode"]
print(word_subsets(w1, ["e", "o"]))
print(word_subsets(w1, ["l", "e"]))
```

### Sample Input:

words1 = ["amazon","apple","facebook","google","leetcode"], words2 = ["e","o"]
words2 = ["l","e"]

### Sample Output:

["facebook", "google", "leetcode"]
["apple", "google", "leetcode"]